ENGINEERING

The Hiring Handbook

Everything about the take-home exercise, how it is scored, and the hour that follows — for the Full-Stack Engineer and QA Engineer roles.

TIME LIMIT	EXPECTED WORK	AFTER THAT
24 hours	5–8 hours	One 60-minute round

SUBMIT TO	ZIP NAMED	PLUS
025pathaksandesh@gmail.com	FirstName_LastName.zip	Repo link + 2–3 min video

A companion to the challenge repository. Where the two disagree the repository is correct — it is the version we keep up to date.

SECTION ONE

What this is, and why it looks like this

We build and maintain web platforms for community organisations — nonprofits, cultural centres, membership associations. The kind of client with real users, real money moving through the system, a board asking questions, and no in-house engineering team at all.

That last part shapes everything. When we ship something broken, nobody at the client can fix it; they call us. So we care about tests and about boring, obvious code more than most teams our size. The team is distributed and overlaps about four hours a day, which makes **written communication a first-class engineering skill here** — and is why the documentation and the video are graded seriously.

ROLE	WHAT YOU WOULD OWN	THE EXERCISE
Full-Stack Engineer	Features end to end — schema, API, UI, tests, deploy — mostly inside a codebase somebody else started. We expect you to push back on a specification when it is wrong, in writing, before you build it.	Build a donation tracking page: backend, frontend, database, monorepo, Docker. A small honest slice of the real work — money, concurrency, authorisation, an admin view, a deploy.
QA Engineer	The definition of *done*. Test strategy, exploratory testing before anything ships, an automated regression suite in CI, and bug reports a developer can reproduce on the first read. QA is not downstream of engineering here; it sits beside it.	We give you a running hall-booking system that we broke on purpose. Find what is wrong, prove it, rank it, and build the net that stops it coming back — roughly what your first month would be.

> **ON THE CLIENT DETAILS**
>
> The organisation described in the challenge repository is fictionalised — the name, domain and numbers are invented, and we are not going to hand a live client's architecture to everyone who applies. **Everything technical is real**: the stack, the constraints, the deadlines and the problems are the ones you would actually walk into.

> **YOU DO NOT HAVE TO READ THE WHOLE REPOSITORY**
>
> There are a lot of files in it, and most are reference. **Required reading is three documents** — the project context, your role's `README.md`, and your role's `REQUIREMENTS.md` — about fifteen minutes. Everything else is linked from the point where it matters, and the root `README.md` has a table saying when to open each one. Do not read ahead; start building.

The constraints that shape the work

- **Money is involved, and it is somebody's donation.** A rounding error is a receipt that does not match a bank statement, found by a volunteer treasurer six weeks later.
- **Concurrency is not theoretical.** A fundraising email goes out and three hundred people hit the same page in ninety seconds.
- **The data is sensitive.** Home addresses, phone numbers, family relationships, giving history. A leak here is harmful, not embarrassing.
- **The audience is bilingual** — English and Nepali. Names do not fit ASCII assumptions, and `toLowerCase()` is not a search strategy.
- **There is no QA department at the client.** If we do not catch it, a donor does.

SECTION TWO

The 24 hours

The clock starts on the email that sent you the repository link. That is a deadline, not a workload — we expect **five to eight hours of actual work**. The rest of the day is so you can sleep and go to your current job. We can tell when somebody pulled an all-nighter, and it does not score better.

> **SCOPE IS YOUR FIRST ENGINEERING DECISION**
>
> Every exercise is split into `MUST` `SHOULD` `STRETCH`. Do the must-haves properly, then decide deliberately what to do with the time left — and write that decision down in your README. We would far rather read "*I stopped here, and here is why*" than find four features that are each seventy percent finished.

If 24 hours does not fit your week, reply and ask for more time *before* the clock runs out. Family, work, illness, a visa appointment — we have never said no to somebody who asked in advance, and it is not recorded against you. What we do notice is somebody going quiet and submitting late. If you simply run out of time, submit what you have with a note on what you would have done next and why. A half-finished exercise with an honest README beats a rushed complete one, and makes a much better interview.

Before you start: what to install

Node.js 24 LTS (`nodejs.org/en/download`, or `nvm install 24`) · **Docker Desktop** or Docker Engine with Compose v2 (`docs.docker.com/get-started/get-docker`) · **Git** (`git-scm.com/downloads`) · a **GitHub account** · a **screen recorder** (Loom, OBS, or `Cmd-Shift-5` on macOS). Confirm with `node -v`, `docker compose version` and `docker run --rm hello-world` *before* you write anything — a Docker daemon that will not start is a thirty-minute problem, and you do not want to meet it at 11pm.

You do not install a database. It runs as a service in your own `docker-compose.yml` and starts with `docker compose up --build` — `postgres:16-alpine` with a named volume and a healthcheck, or SQLite with a named volume and no `db` service at all. We will not create a cloud database or install Postgres to review your work, and a free-tier hosted URL will be expired or rate-limited by the time we open it. Both compose shapes are written out for you in the repository.

What a good six-hour submission looks like

So you can calibrate rather than guess. Both of these are **strong** submissions, not minimum ones — each has scored in the eighties.

Full-stack

A monorepo with one genuinely shared package · migrations and the seed loaded with the totals correct · a public campaign page with privacy enforced in the query · a donation form that validates on the server and returns a receipt number · an admin login, a searchable list, and a refund that corrects every total · three tests — money boundaries, authorisation, refund · `docker compose up` working from a clean clone · a README with *Decisions* and *Known issues* · fourteen commits and a three-minute video.

No stretch goals. No CSV export. No pagination. No bilingual toggle.

QA

A two-page test plan with a real out-of-scope list and a risk ranking · **eight** ranked bug reports, two or three of them serious and at least one not visible in the interface · **nine** automated tests — six API, one concurrency, one UI flow, one clearly failing and labelled with the bug it covers · a summary ending in a go / no-go recommendation with conditions · nine commits and a three-minute video.

No CI workflow. No load testing. No cross-browser matrix. Eight bugs, not twenty-five.

The submissions that score below sixty are almost never the ones that ran out of time. They are the ones that started the stretch goal before the refund worked, or shipped twenty green UI tests with no ranking and no

recommendation.

A sensible shape for the working day

HOUR	WHAT TO DO
`0:00–0:30`	**Read before you type.** The requirements and the specification, properly. Write down the schema you intend, the decisions you already know you must make, and what you are *not* going to build.
`0:30–1:00`	**Tooling first.** Linter, formatter and test runner configured before any feature code exists. Retrofitting them is how standards get skipped, and formatting churn at the end hides your actual work.
`1:00–2:15`	**The foundation.** Full-stack: schema, migrations, seed, and the money type with its tests. QA: exploratory testing with notes, evidence captured as you go.
`2:15–5:30`	**The work.** One piece at a time, verifying each before building the next on top of it. That single habit is the difference between a calm day and a bad night.
`5:30–6:30`	**Docker, then the write-up.** Do not leave Docker to the last thirty minutes — it is the largest scoring category and the most common way a good submission loses twenty points.
`6:30–7:30`	**Clean-clone test, then record.** Clone your own repo into a fresh directory and follow your own README word for word, using nothing that is only in your head. Fix what breaks. Record in one take.

SECTION THREE

How it is judged

Two people read every submission independently, score it against the sheet below, then compare; where we differ by more than a band a third person reads it. We publish the rubric because an exercise where the candidate has to guess the criteria tests guessing, not engineering.

#	CATEGORY	FULL-STACK	QA	WHAT EARNS THE POINTS
1	**It runs**	20	20	A stranger clones it, follows your README, and it works. First try.
2	**Core requirements**	25	30	The must-have list, done properly. For QA: what you found, and how well you reported it.
3	**Engineering judgement**	20	15	Structure, naming, the trade-offs you made, and what you chose *not* to build.
4	**Security & data integrity**	15	15	Money, authorisation, validation, concurrency, secrets.
5	**Documentation & commits**	10	10	README, comments, commit history, bug reports.
6	**Video walkthrough**	10	10	Can you explain your own work in three minutes.

BAND	WHAT HAPPENS
80+	Strong hire. Interview scheduled, and the conversation is about how you would approach the real work.
65–79	Interview. We have specific questions about specific decisions.
50–64	Borderline — usually one category collapsed. We may ask you to talk us through it first.
Under 50	No, with the specific reason. Not "we went with another candidate".

There is also a bonus of up to **+8** for stretch work. It applies only once the must-haves are complete, and it cannot rescue a submission that lost category one.

The four things that sink most submissions

01 **It does not run.** One honest attempt: clean clone, follow your README, no improvisation. If we cannot resolve an error in ten minutes, category one is zero and everything after it is guesswork. Twenty minutes of clean-clone rehearsal prevents it.

02 **One giant commit.** `git log` shows a single commit called "final" containing four thousand lines. We cannot see how you work, which is half of what we were trying to learn.

03 **Money as a float.** `parseFloat`, a total of `149.99999999999997`, a `REAL` column. In a donation system this alone is disqualifying — it would mean checking every arithmetic line you ever write.

04 **No authorisation check.** Authentication answers "who are you". Authorisation answers "may you touch *this* record". Plenty of submissions have a login and no second check.

What earns disproportionate credit

A written trade-off

A *Decisions* section saying "*I used optimistic locking rather than a row lock because the contention window is short; under heavier write load I would switch*" moves you a whole band. It shows the thing an interview is trying to find out.

Saying what is broken

"*The CSV export does not escape commas in names. I found it, I ran out of time, here is where.*" We find these anyway; the difference is whether we found it *with* you or *about* you.

Disagreeing with the spec

These specifications have deliberate gaps and at least one arguable decision. Noticing and saying so, in writing, is exactly what we want.

Tests on the hard part

One test proving two concurrent requests cannot double-book the same unit tells us more than forty assertions on a form validator.

Restraint

Must-haves done excellently and stretch goals skipped with a sentence beats everything attempted at sixty percent.

> **WHAT WE ARE NOT TESTING**
>
> Not pixel-perfect design — legible is enough and there is no Figma file. Not payment integration — the client's payments already work and are out of scope. Not breadth of buzzwords — a boring, well-tested Express and Postgres service outscores a half-working microservice mesh. And not whether you used AI; we assume you did.

SECTION FOUR

What we mean by code quality

These are the standards we hold ourselves to in review. It would be unfair to grade you against a bar we kept to ourselves.

Readable over clever

Write the version a reviewer understands on the first pass — if a line needs a paragraph to justify itself, it is the wrong line. Descriptive names. Early returns over nested conditionals. One responsibility per function: if the description needs the word "and" twice, split it. No premature abstraction — two call sites is not yet a pattern. **And no source file over 500 lines**; when one approaches it, split along a real seam, by responsibility rather than by cutting it in half.

Comments explain *why*. The code already says *what*.

```
// Bad — restates the line below it, and ceremony on a self-evident function
// increment the counter
counter += 1;

// Good — records a decision the reader cannot infer
// Amounts are integer cents everywhere below this line. The API accepts
// dollars as a string and converts once, here, so no float ever touches
// the arithmetic.

// Good — flags a thing that looks wrong and is not
// We deliberately re-read the hold inside the transaction even though we
// just read it above. The first read is for the 404; this one is for the race.

// Good — names the constraint behind a magic number
// Ten minutes, from the change order. Do not shorten it without asking —
// donors on mobile need the time to finish the card form.
const HOLD_DURATION_MS = 10 * 60 * 1000;
```

Most functions need no comment at all; a file with a comment on every third line is as hard to read as one with none. Comment the decisions, the invariants, and the parts that look wrong but are deliberate. A precise type or schema is worth more than a paragraph and cannot go stale — prefer it.

Tests are part of the deliverable

No coverage threshold, and we will not count your percentage. We look at *what* you chose to test: the happy path, **every error branch you actually wrote**, boundary values — empty, one, maximum, one over maximum — and any invariant your design depends on. If a comment claims something is always true, a test should prove it. And test before proceeding: never build the next piece on an unverified one. Over a single day that is faster, not slower.

Your commit history is graded

The code tells us what you can produce in a day with no reviewer; the history tells us *how you got there* — whether you work in coherent steps, keep the tree working, and would be pleasant to review. Every change here goes through a pull request a colleague reads at 7am in another timezone.

Good

```
Add the donation intake endpoint
Store amounts as integer cents
Fix the progress bar counting refunds
Move the seed script out of the entrypoint
Add a concurrency test for unit holds
```

Bad

```
final
wip
Updated files
feat: implement comprehensive donation
     management functionality
```

Roughly ten to thirty commits for an exercise this size. Thirty tiny honest commits is fine; six well-shaped ones is fine; one is not. Use a body only when it earns one, to explain *why*. If you already use Conventional Commits keep using them — if you do not, please do not adopt them here; we would rather see your natural habits.

> **PLEASE DO NOT RECONSTRUCT A HISTORY AT THE END**
>
> A fabricated history is visible: timestamps in an unnaturally regular rhythm, no fix-up commits, no commit that reverses an earlier decision, files appearing complete rather than growing. Real history contains a commit that undoes something — that is a *good* sign, because it means you changed your mind, which is what engineering is.

Documentation

Your root README is the most important file in your submission. We read it first and run your project from it without deviating. It must contain: what this is · your stack and why · how to run it, by Docker and locally, both

tested · **what we should see when it works** · test accounts in plain text · how to run the tests and what currently passes · a *Decisions* section · a *Known issues* section · an API reference · your video link. Any folder a reader would otherwise reverse-engineer gets a three-sentence README answering: what is this, how do I run it, what should I know before changing it.

SECTION FIVE

The security baseline

The platform you would be joining holds home addresses, phone numbers, family relationships and giving history for several thousand people, and it moves their money. Security is not a checklist item in this domain; it is the domain. Full-stack candidates must implement this; QA candidates must test for its absence, because several of the planted defects are security defects.

The seven non-negotiables

01 **No secrets in the repository, or in its history.** Commit `.env.example` with placeholders, never `.env`. Removing a key in a later commit does not remove it — the history is what we read. Test-account passwords in your README are fine; those are fixtures.

02 **Every endpoint answers "who are you" *and* "may you touch this record".** Put the ownership rule in the query, not in a branch after the fetch. Return **404, not 403**, for a record that exists and is not yours — a 403 confirms it exists, which lets somebody enumerate your donors.

03 **Validate every input at the boundary, with a schema.** Reject unknown fields rather than passing them to your ORM. Never trust a client-supplied amount, price, total, status or role — look the price up on the server.

04 **Money is integer minor units.** Cents, as integers, in the database, the API and your logic. Never `FLOAT` or `REAL` on a monetary column. Format once, at the display edge. Store the currency code beside the amount even with one currency.

05 **Parameterised queries, always** — including the dynamic `ORDER BY` built from a query parameter. Sort columns come from a fixed allow-list, never from the request.

06 **Escape output; assume every stored string is hostile.** A dedication message is rendered on a public page, in an admin table, in a PDF and in an email — four contexts, four escapes. Do not reach for `innerHTML` or `dangerouslySetInnerHTML` to make something render "properly".

07 **Do not leak internals in errors.** Log the stack on the server; send the client a stable code and a human sentence. Keep failed-login responses identical whether the account exists or not, or your login form is a user-enumeration endpoint.

Also expected, and part of the score

Passwords hashed with bcrypt, argon2 or scrypt — never a bare SHA. Session tokens that are random and expire; a token derived from an email is a forgeable credential. `HttpOnly`, `Secure`, `SameSite` cookies. Rate limiting on login and on donation intake — an unlimited donation endpoint is a card-testing target, and the client pays a fee for every declined attempt. Security headers. CORS that names origins. `npm audit --omit=dev` run before you submit, so you know the number.

Data integrity, which is security in this domain

- **Concurrency safety on anything reserved or limited.** Check-then-act across two statements is a race. A row lock, a unique constraint, an atomic conditional update, or optimistic concurrency with a version column — any of them, chosen deliberately and explained in a comment.
- **Idempotency on writes that move money.** A double click, a mobile retry or a redelivered webhook must not create two donations.

- **Refunds reflected in every aggregate**, in the same operation — not on a nightly job.
- **Timestamps in UTC**, converted for display. "Today" means today in the organisation's timezone, defined in exactly one documented place.

> **NAMING A RISK COUNTS**
>
> Real payment processing, OAuth, MFA, encryption at rest and infrastructure hardening are all out of scope. If you notice one of them matters and have not built it, write a line about it in your README. Naming a risk you chose not to mitigate scores nearly as well as mitigating it, and it is exactly what we would want on a Tuesday afternoon with a deadline.

SECTION SIX

Using AI coding agents

You may. We do. Pretending otherwise would make this a test of whether you can hide something. Claude Code, Cursor, Copilot, Windsurf, Codex, Gemini CLI, Aider, Cline — any of them, plus any chat assistant, documentation, blog post or book. The only thing not allowed is submitting somebody else's completed solution as your own.

> **THE ONE RULE**
>
> **You must be able to explain and defend every line you submit, live, without notes.** In the interview we will open a file — often one we suspect you did not type — and ask why it is that way, what happens if the input is empty, and what breaks if we delete a line. "*The AI wrote that part*" ends the interview. Not because you used AI, but because you shipped code you had not read, and in this domain that is how a donor's address ends up in a public JSON response.

The person we want used an agent to move four times faster, then read every line it produced, deleted a third of it, and can argue about the rest.

Required: `AI-USAGE.md`

Half a page at the root of your submission: which tools, roughly how much, **what you changed about what it produced**, and what you deliberately did not let it near. That middle part is one of the strongest signals in the whole submission — it shows your review instincts, which is most of what senior engineering is now. An honest file saying "I used it for almost everything" is fine. A missing one is not.

Where agents reliably let people down

AREA	WHAT THE AGENT TENDS TO PRODUCE
Money	`parseFloat`, `toFixed(2)`, float columns. Wrong in a donation system.
Concurrency	Read, check, then write, in two statements, with no transaction. The exact race.
Authorisation	An auth middleware, and then no per-record ownership check.
Errors	`catch (e) { res.status(500).json({ error: e.message }) }` — leaking internals.
Timezones / Unicode	`new Date()` with no zone, so "today" is wrong for half your users; ASCII assumptions, so Devanagari search silently returns nothing — or everything.
Tests	Many assertions on trivia, none on the hard part. Mocks that assert the mock.
Docker	Single stage, running as root, `node_modules` from the host, no health check.
Comments	`// Step 3: create the handler`, `// This ensures robust error handling`. Noise — delete it.
Consistency	Three naming conventions and two error shapes, because each file was generated in isolation.

That table is, more or less, the review checklist we would use on your first pull request; running it over your own work before you submit is free points. **Read your whole diff before you commit** — not skim, read. That habit is the actual skill we are hiring for, and it is visible from across the room.

QA candidates: generated suites are notorious for asserting the mock rather than the behaviour. Check that each of your tests **fails when the bug is present** — a regression test that has never been red is not a regression test. And do not let an agent write your bug reports unedited; "the application exhibits unexpected behaviour" is worthless to a developer.

SECTION SEVEN

Where and how to send it

01 **Fork the challenge repository.** The fork graph tells us exactly which commits are yours. If your account cannot fork it, clone it, delete `.git`, start fresh with one baseline commit called "*Import the challenge repository as provided*", and say in your README which route you took.

02 **Work in it, committing as you go.** On `main`, or on a branch with a pull request into your own `main` — a PR with a real description is a small bonus, because it is the best writing sample you can give us.

03 **Write your own README at the root.** Ours describes the challenge; yours must describe your submission. A stranger must be able to run your project from it without asking you anything. Add `AI-USAGE.md` beside it.

04 **Make it reachable.** Public, or private with **@sandesPathak** invited. If it is private and you forget the invitation we email you once; unanswered for a day and we grade the zip alone, which means we cannot see your history.

05 **Record the video.** Two to three minutes. Loom, YouTube unlisted, Drive set to "anyone with the link", or Vimeo — anywhere we can open without an account. **Test the link in a private browser window.** A "Request access" screen is the commonest way a good submission stalls.

06 **Make the zip, named exactly** `FirstName_LastName.zip` — capital first letters, one underscore, no spaces, no dates, no `v2`. For example `Anisha_Gurung.zip`.

07 **Send the email** to **025pathaksandesh@gmail.com**, with the subject line copied exactly.

Building the zip

```
# from the folder above your project
zip -r Anisha_Gurung.zip my-project \
  -x "*/node_modules/*" "*/.next/*" \
     "*/dist/*" "*/coverage/*" "*/.env"

unzip -l Anisha_Gurung.zip | head -40
du -h Anisha_Gurung.zip     # under ~50 MB
```

Include `.git` — small once `node_modules` is out, and it is how we read your history if the repo link breaks. **Exclude** `node_modules`; a 400 MB zip may bounce off our mailbox. Over 50 MB, send a Drive link rather than deleting half your project to make it fit. And no real secrets.

Subject line — copy one exactly

```
Full-Stack Challenge — FirstName LastName
QA Challenge — FirstName LastName
```

Body — this is enough

```
Hi,

Here is my submission for the
<Full-Stack / QA> Engineer challenge.

GitHub: https://github.com/<you>/<repo>
Video:  <link>
Zip:    attached

Roughly <N> hours. Node <version>,
<package manager>, <database>.

Notes:
- <what you skipped, what you would do
  next, anything broken>

Thanks,
<Name> · <phone, timezone>
```

BEFORE YOU HIT SEND

Repo forked or imported with a clean baseline · more than one commit with human messages · your own README explains how to run it · `AI-USAGE.md` present · `.env.example` committed and no real secret anywhere in the history · `docker compose up` works from a clean clone · tests run and the README says which pass · the role checklist complete · video is 2–3 minutes and opens privately · zip named correctly, excludes `node_modules`, includes `.git` · repo public or `@sandesPathak` invited · subject line matches exactly.

We acknowledge on arrival and reply within **three working days**, either way. If you have not heard by then, email again — it means something went to spam, not that you were rejected quietly.

SECTION EIGHT

The hour that follows

If we move forward, the next step is a **single round of about sixty minutes**: a code walkthrough of your own submission, then a behavioural conversation. Two of us — the technical lead, and one engineer or QA engineer who would work beside you. **There is no separate algorithm round**, no whiteboard, no take-two. Your submission is the technical assessment; inventing a second artificial test after you have shown us real work would be disrespectful of your time.

TIME	WHAT HAPPENS
`0:00–0:05`	**Hello.** Who we are, what the work looks like right now, how the call will run. Nothing is assessed. If you are nervous, this is where you stop being nervous.
`0:05–0:35`	**Code walkthrough.** You share your screen and open your own submission. You drive five minutes — the shape of it and the part you are proudest of. We drive ten, opening two or three files we flagged. Then a small live change, in your own codebase and editor, with your usual tools **including your AI agent if that is how you work**.
`0:35–0:55`	**Behavioural.** Three or four questions with follow-ups, about situations that actually arise here. Not a personality quiz.
`0:55–1:00`	**Your questions.** Please have some. We answer honestly, including about the annoying parts of the job — and there are some.

The kind of thing we ask in the walkthrough

- "Walk me through everything between the donor clicking Donate and the row landing in the database."
- "Two people click the same unit in the same millisecond. Show me in your code why only one gets it." — then: "What if there were two instances of your API behind a load balancer?"
- "Why integer cents rather than a decimal column? Where does the conversion happen, and how many places *could* it happen?"
- "This endpoint checks the session. What stops me reading somebody else's donation?"
- "What breaks first at four thousand donations a day instead of forty?" · "You skipped this — talk me through that decision."
- "Here is a bug we found in your submission. No trap — what would you do about it?"

For QA: "How did you find the one that needed two requests at once?" · "You ranked this second — convince me it is not first." · "A developer says finding four is expected behaviour. What do you do?" · "The release is in four hours and you have just found this. Go."

The live change is fifteen or twenty minutes, small and realistic: *add a minimum donation amount for one campaign and a test for it*, or, for QA, *this bug was fixed — write the regression test that would have caught it*. We are not watching whether you type fast. We are watching how you orient in your own code, whether you run the tests, whether you read the error message or guess, and whether you talk while you think.

The behavioural half — the follow-up is where the answer is

QUESTION	WHAT WE ARE LISTENING FOR
A bug you put into production — what happened, how did you find out, what did you do in the first hour?	Ownership without self-flagellation, and a **systemic** fix rather than "I was more careful after that".
A requirement you thought was wrong. What did you do?	Pushed back early, in writing, with a reason — then committed fully once decided, including when it went against you.
Three hours in, stuck, everyone who could help is asleep.	How long you thrash before you write the question down. No single right answer.
Scope will not fit the date. Who do you talk to, and what do you say?	That you raise it early, with options and a recommendation, not as a complaint.
Only person awake; the client says the donation page shows the wrong total on a fundraising day.	Triage instinct. What you check first, and when you tell somebody.
How do you tell a colleague their pull request has a security problem? And how do you take that yourself?	Directness with care, and no defensiveness in the reverse direction.

No trick. The best answers are specific, short, and include the part that did not go well. The weakest are hypothetical — "*I would always communicate proactively*" — because we cannot tell whether you have ever actually done it.

How the hour is scored

Ownership of your code	30%
Technical reasoning	25%
The live change	15%
Communication	15%
Collaboration & judgement	15%

Say "I don't know" when you do not know, then say how you would find out. That scores better than a confident wrong answer, every time.

How to prepare — half an hour is plenty

- Re-read your own submission, including the commit history
- Have it running *before* the call, so we are not watching `npm install`
- Prepare a ninety-second "here is what I built and here is the interesting part"
- Have three concrete stories: a failure, a disagreement, something you are proud of
- Write down two questions for us

Do not rehearse the behavioural answers word for word — we can hear it, and it makes you sound less impressive than you are.

> **ACCOMMODATIONS**
>
> If anything about this format is a barrier — questions in advance, more time in the live segment, captions, screen-sharing difficulties, or answering the behavioural part in writing — **say so when we schedule**. We will adjust it, it is not held against you, and you do not have to explain why.

A decision within two working days. If it is a no, we tell you the specific thing you would need to change — not "we went with another candidate". You are welcome to reapply after three months, and we have hired people who did exactly that.

SECTION NINE

Good luck

Do the must-haves properly

Then stop, and write down why you stopped. Nobody has ever been rejected for skipping a stretch goal with a good sentence attached.

Rehearse the clean clone

Twenty minutes, and it protects the single largest category on the sheet. Follow your own README as though you had never seen the project.

Be honest in the README

Every submission has gaps. We find them either way. The only question is whether we found them with you or about you.

Record the first take

A stumble, a "sorry, let me find that file", a dog barking — none of it costs you anything. Three polished minutes and five raw ones score identically.

Ask if you are stuck

A good clarifying question is a point in your favour. Reply with `[Challenge Question]` in the subject; we answer within a working day.

And close the laptop

The honest submission you finished is worth more than the perfect one you did not.

We know what a take-home costs, especially with a full-time job and a life attached. We have tried to make this one worth the hours: the rubric is published rather than guessed at, the exercise is a real slice of the actual work rather than a puzzle, and the interview is a conversation about what you built rather than a test we invented afterwards.

Whatever happens, you leave with something you can put in your portfolio and a specific answer about why — and that is the least we owe you for a day of your time.

We are looking forward to reading it. Good luck.

Sandesh Pathak

Technical Lead · 025pathaksandesh@gmail.com